Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Владимирский государственный университет
имени Александра Григорьевича и Николая Григорьевича Столетовых»
(ВлГУ)
Кафедра ФиПМ

ЛАБОРАТОРНАЯ РАБОТА №3

по дисциплине «Операционные системы»

на тему:

«УПРАВЛЕНИЕ СИСТЕМНЫМИ РЕСУРСАМИ СРЕДСТВАМИ SHELL-ИНТЕРПРЕТАТОРА. Компилятор GCC. Оптимизация»

Выполнил:
ст. группы ИТ-124
Горностаев А.А.
Принял:
ассистент каф. ФиПМ
Болачков А. В.

Владимир 2026

Цель работы.

Изучить программные средства языка Shell (структура команды, группирование команд, перенаправление ввода-вывода, конвейер команд, Shell-переменные, макроподстановка результатов в Shell-командах, программные конструкции).

Постановка задачи.

Часть 1.

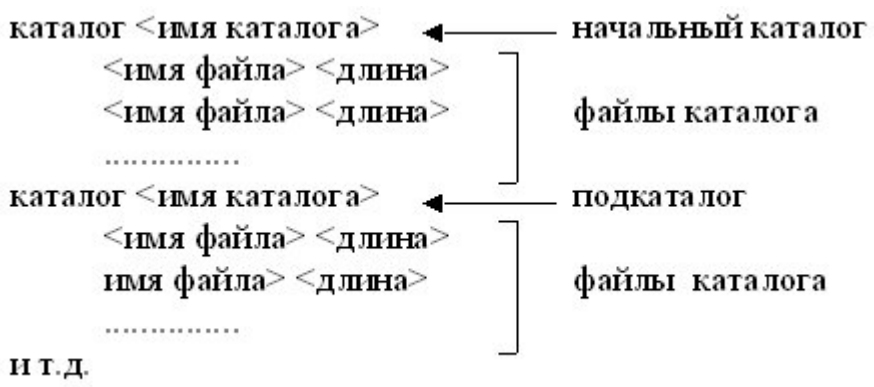
Задание 1. Напишите сценарий, использующий оператор case, для поддержки выполнения основных математических операций: + сложение - вычитание x умножение / деление Сценарий работает следующим образом: $ ./script.sh 20 / 3 Также проверьте число аргументов командной строки.

Задание 2. Напишите сценарий, определяющий, существует ли указанный файл. Имя файла указывается, как параметр командной строки, проверьте его наличие.

Задание 3. По варианту.

Варианты задания 3.

1. Shell-программа подсчитывает количество и выводит список всех файлов (без каталогов) в порядке уменьшения их длин в поддереве, начиная с каталога, имя которого задано параметром Shell-программы. Форма вывода результата:



3. Задание варианта 1, но список файлов в каталоге выводится в алфавитном порядке.

Часть 2.

1. Выполнить продемонстрированные в части 2 примеры.

2. На основе примера, демонстрирующего различные уровни оптимизации, написать сценарий, выполняющий следующие действия:

● Вычисление времени выполнения программы.

● Вычисление занимаемого исполняемым файлом дискового пространства (в байтах).

Сценарий должен принимать следующие параметры командной строки:

● Имя исходного файла программы.

● Требуемые ключи оптимизации, если таковые нужны.

Вывод программы должен содержать следующую информацию:

● Текущие ключи оптимизации

● Время затраченное программой на выполнение

● Занимаемое программой дисковое пространство

Теоретическая часть

Часть 1. Сценарии командного процессора

Обычно в ОС UNIX доступны несколько интерпретаторов. Наиболее распространены Bourne-shell (или просто - shell), C-shell, Korn-shell. В идейном плане все эти интерпретаторы близки и в дальнейшем речь будет идти о стандартном Shell (/bin/sh).

Работая на командном языке, пользователь может вводить переменные, присваивать им значения, выполнять простые команды, строить составные команды, управлять потоком выполнения команд, объединять последовательность команд в процедуры (командные файлы). На уровне командного языка доступны такие свойства системы как соединение процессов через программный канал, направление стандартного ввода/вывода в конкретные файлы, синхронное и асинхронное выполнение команд.

Если указанный интерпретатору файл является текстовым и содержит команды командного языка (командный файл) и при этом имеет разрешение на выполнение (помечен "x"), Shell-интерпретатор интерпретирует и выполняет команды этого файла. Другой способ вызова командного файла - использование команды sh (вызов интерпретатора), в котором первым аргументом указывается имя командного файла.

Коротко перечислим средства группирования команд и перенаправления ввода/вывода:

- cmd1 arg ...; cmd2 arg ...; ... cmdN arg ... - последовательное выполнение команд;
- cmd1 arg ...& cmd2 arg ...& ... cmdN arg ... - асинхронное выполнение команд;
- cmd1 arg ... && cmd2 arg ... - зависимость последующей команды от предыдущей таким образом, что последующая команда выполняется, если предыдущая выдала нулевое значение;
- cmd1 arg ... || cmd2 arg ... - зависимость последующей команды от предыдущей таким образом, что последующая команда выполняется, если предыдущая выдала ненулевое значение;
- cmd > file - стандартный вывод направлен в файл file;
- cmd >> file - стандартный вывод направлен в конец файла file;
- cmd < file - стандартный ввод выполняется из файла file;
- cmd1 | cmd2 - конвейер команд, в котором стандартный вывод команды cmd1 направлен на стандартный вход команды cmd2.

Shell-переменные могут хранить строки текста. Правила формирования их имен аналогичны правилам задания имен переменных в обычных языках программирования. При необходимости присвоить Shell-переменной значение, содержащее пробелы и другие специальные знаки, оно заключается в кавычки. При использовании Shell-переменной в выражении ее имени должен предшествовать знак $. В последовательности символов те из них, которые составляют имя, должны быть выделены в { } или " ". Кроме того интерпретатор Shell автоматически присваивает значения пяти своим переменным:

$? - значение, возвращаемое последней выполняемой командой;

$$ - идентификационный номер процесса Shell;

$! - идентификационный номер фонового процесса, запускаемого интерпретатором Shell последним;

$# - число аргументов, переданных в Shell;

$- - флаги, переданные в Shell.

Для отмены специальных символов ($,|,пробел и т.д.) в Shell-программах существуют следующие правила:

если символу предшествует обратная косая черта, то его специальный символ отменяется;

отменяется специальный смысл всех символов, вошедших в последовательность, заключенную в апострофы.

При вызове Shell-программ им могут передаваться параметры. Соответствующие аргументы в Shell-программах идентифицируются $1, $2, $3 и т.д. Кроме того, переменная $0 соответствует имени выполняемой Shell-программы, а переменная $# - числу аргументов в команде.

Shell-интерпретатор дает возможность выполнять подстановку результатов выполнения команд в Shell-программах. Если команда заключена в одиночные обратные кавычки, то интерпретатор Shell выполняет эту команду и подставляет вместо нее полученный результат.

Наиболее важные команды для составления Shell-программ:

- команда echo выводит в выходной поток значения своих аргументов;
- команда expr выполняет арифметические действия над своими аргументами;
- команда eval обеспечивает дополнительный уровень подстановки своих аргументов, а затем их выполнение;
- команда test с соответствующими ключами проверяет необходимое условие;

- команда sleep служит для реализации задержки.

Программные конструкции Shell-программ:

Условный оператор if:

```
if if_list
    [then then_list
elif elif_list]
    then then_list
[else else_list]
fi
```

Циклы while и until:

```
while while_list
            do do_list
done
until until_list
        do do_list
done
```

Цикл for

```
for name [in word1, word2...]
   do do_list
done
```

Структура case

```
case word in
        pattern1) part_list;;
        pattern2) part_list;;
esac
```

Описание шагов необходимых для написания и исполнения сценария командного процессора:

Шаг 1: Откройте файл используя текстовый редактор (например "gedit" или "nano".)

nano firstshellscript.sh

Шаг 2: Все сценарии должны начинаться со строки "#!/bin/bash" или с указания любого другого командого процессора, который вы предпочитаете. Эта строка называется shebang, и хотя она выглядит как комментарий, это не так: она уведомляет оболочку о том, какой командный процессор должен использоваться для этого сценария. Указанный путь должен быть абсолютным (вы не можете просто написать "bash", к примеру), а shebang должен находиться на первой строке скрипта без символов перед ним.

Шаг 3: Напишите код, который вы хотите выполнить. Наш первый сценарий будет обычной программой "Hello World", которую мы разместим в файле названном 'firstshellscript.sh'.

```
#!/bin/sh

echo "Hello World"
```

Шаг 4: Следующий шаг - сделать сценарий исполняемым, используя команду "chmod".

```
chmod +x firstshellscript.sh
```

Шаг 5: Выполнить сценарий. Это может быть сделано вводом имени сценария в командную строку с указанием его пути. Если сценарий находится в текущей директории, это очень просто:

```
bash$ ./firstshellscript.sh

Hello World
```

Если вы хотите увидеть выполнение пошагово, что очень полезно для отладки - тогда выполните сценарий с опцией '-x' (что означает 'раскрыть аргументы'):

```
sh -x firstshellscript.sh

+ echo 'Hello World'

Hello World
```

Чтобы увидеть содержание сценария вы можете использовать команду 'cat' или просто открыть сценарий в любом текстовом редакторе:

```
bash$ cat firstshellscript.sh

#!/bin/sh

echo "Hello World"
```

Комментарии в сценарии

В сценариях командного процессора все строки, начинающиеся с #, являются комментариями.

Это строка комментария.

Это еще одна строка комментария.

Вы также можете написать многострочные комментарии, используя двоеточие и одинарные кавычки:

: 'Это комментарий.

Это снова комментарий.

Кто бы мог подумать, что это еще один комментарий.'

Примечание: Это не будет работать, если в комментариях будут символы одинарной кавычки.

Переменные

Переменные являются самой значительной частью любого языка программирования, будь это Perl, C, или сценарии. В сценариях переменные разделяются на системные и пользовательские.

Системные переменные

Системные переменные определяются и хранятся в окружении родительского командного процессора (командного процессора, из которого ваш сценарий выполняется). Они также называются переменными окружения. Имена этих переменных состоят из заглавных букв, и могут быть показаны командой 'set'. Примерами системных переменных являются PWD, HOME, USER. Значения этих системных переменных могут быть показаны по отдельности командой 'echo'. Например 'echo $HOME' выведет значение, хранящееся в системной переменной HOME. Когда устанавливаете системную переменную, не забудьте использовать команду 'export', чтобы

сделать ее доступной дочерним командным процессорам (любые командные процессоры, которые запущены из текущего, включая сценарии):

bash$ SCRIPT_PATH=/home/blessen/shellscript

bash$ export SCRIPT_PATH

Современные командные процессоры также позволяют сделать это одной командой:

bash$ export SCRIPT_PATH=/home/blessen/shellscript

Пользовательские переменные

Эти переменные часто используются в сценариях, когда нет необходимости или желания делать переменные доступными другим программам. Их имена не могут начинаться с цифр, записываются в нижнем регистре и вместо пробела используется знак подчеркивания - например "define_tempval". Когда мы присваиваем значение переменной, мы пишем имя переменной перед знаком равенства '=', за которым немедленно идет значение, например 'define_tempval=blessen' (заметьте, не должно быть пробелов перед или за знаком равенства.). А чтобы использовать или показать значение "define_tempval", мы должны использовать команду 'echo' со знаком '$' перед именем переменной, например:

bash$ echo $define_tempval

blessen

Следующий сценарий устанавливает переменную "username" и показывает ее значение.

#!/bin/sh

username=blessen

echo "username - $username"

Аргументы командной строки

Это переменные, которые содержат аргументы выполняемого сценария. Доступ к этим переменным может быть получен через имена $1, $2, ... $n, где $1 первый аргумент командной строки, $2 второй, и так далее. Аргументы располагаются после имени сценария и разделены пробелами. Переменная $0 - это имя сценария. Переменная $# хранит количество аргументов командной строки, это количество ограничено 9 аргументами в старых шеллах и практически неограниченно в современных.

Рассмотрим сценарий, который возьмет два аргумента командной строки и покажет их. Мы назовем его 'commandline.sh':

```
#!/bin/sh

echo "Первый агрумент - $1"

echo "Второй агрумент - $2"
```

Если запустить 'commandline.sh' с аргументами командной строки "blessen" и "lijoe" результат будет таким:

```
bash$ ./commandline.sh blessen lijoe

Первый аргумент - blessen

Второй аргумент - lijoe
```

Переменная кода возврата

Эта переменная сообщает нам, была ли последняя команда успешно выполнена. Она обозначается $?. Нулевое значение означает, что команда была успешно выполнена. Любые другие числа означают, что команда была выполнена с ошибкой (также некоторые программы, такие как 'mail', используют ненулевое значение возврата для отображения состояния, а не ошибки). Таким образом, это очень полезное свойство при написании сценариев.

Чтобы проверить работу переменной кода возврата, создайте файл с именем "test", выполнив команду `touch test` . Затем покажите содержание файла:

bash$ cat test

Проверьте значение $?.

bash$ echo $?

0

Значение равно нулю, потому что команда была успешно выполнена. Теперь попробуйте команду 'cat' с несуществующим файлом:

bash$ cat xyz1

bash$ echo $?

1

Значение 1 показывает, что предыдущая команда была выполнена с ошибкой.

Практическая часть

Часть 1. Сценарии командного процессора

Задание 1. Калькулятор с оператором case

Создаём скрипт math.sh с помощью команды nano

```
alexander@alexander-PC:~/Документы/OS$ cd laba3
alexander@alexander-PC:~/Документы/OS/laba3$ nano math.sh
```

```
  GNU nano 7.2                                math.sh
#!/bin/bash

if [ $# -ne 3 ]; then
        echo "Ошибка: требуется 3 аргумента (число, оператор, число)"
        exit 1
fi

num1=$1
op=$2
num2=$3

case $op in
        +)
                result=$(echo "$num1 + $num2" | bc)
                ;;
        -)
                result=$(echo "$num1 - $num2" | bc)
                ;;
        \*|x)
                result=$(echo "$num1 * $num2" | bc)
                ;;
        /)
                if [ "$num2" = "0" ]; then
                        echo  "Ошибка: деление на ноль"
                        exit 1
                fi
                result=$(echo "scale=2; $num1 / $num2" | bc)
                ;;
        *)
                echo "Неизвестный оператор. Используйте +, -, x(\*), /"
                exit 1
                ;;
esac

echo "Результат: $result"
```

```
alexander@alexander-PC:~/Документы/OS/laba3$ ./math.sh 20 / 3
Результат: 6.66
alexander@alexander-PC:~/Документы/OS/laba3$ ./math.sh 2 x 15
Результат: 30
alexander@alexander-PC:~/Документы/OS/laba3$ ./math.sh 1.2 / 0.6
Результат: 2.00
```

```
alexander@alexander-PC:~/Документы/OS/laba3$ ./math.sh 2.3 + 2.1
Результат: 4.4
alexander@alexander-PC:~/Документы/OS/laba3$ ./math.sh 4.1 - 2.1
Результат: 2.0
```

```
alexander@alexander-PC:~/Документы/OS/laba3$ ./math.sh 20 / 0
Ошибка: деление на ноль
```

math.sh – калькулятор с оператором case.

Проверяет, что передано ровно три аргумента (число, оператор, число). В зависимости от оператора (+, -, x(*), /) выполняет соответствующую арифметическую операцию через bc (для поддержки дробей). При делении проверяет деление на ноль. Для неизвестного оператора выводит ошибку.

Задание 2. Проверка существования файла

```
alexander@alexander-PC:~/Документы/OS/laba3$ nano checkfile.sh
```

```
  GNU nano 7.2                                    checkfile.
#!/bin/bash

if [ $# -ne 1 ]; then
    echo "Использование: $0 <имя_файла>"
    exit 1
fi

file="$1"

if [ -e "$file" ]; then
    echo "Файл '$file' существует."
    if [ -f "$file" ]; then
        echo "Это обычный файл."
    elif [ -d "$file" ]; then
        echo "Это каталог."
    else
        echo "Это специальный файл."
    fi
else
    echo "Файл '$file' не существует."
fi
```

```
alexander@alexander-PC:~/Документы/OS/laba3$ ./checkfile.sh ../laba6
Файл '../laba6' не существует.
```

```
alexander@alexander-PC:~/Документы/OS/laba3$ ./checkfile.sh /etc/passwd
Файл '/etc/passwd' существует.
Это обычный файл.
```

checkfile.sh – проверка существования файла.

Принимает один аргумент – имя файла. С помощью условного оператора -e проверяет, существует ли файл. Если существует, определяет его тип (обычный файл, каталог или другое) с помощью флагов -f и -d. Выводит соответствующее сообщение.

Задание 3 (Вариант 3). Список файлов в алфавитном порядке с размерами

```
alexander@alexander-PC:~/Документы/OS/laba3$ nano listfiles.sh
```

```
  GNU nano 7.2                                listfiles.sh
#!/bin/bash

if [ $# -ne 1 ]; then
    echo "Использование: $0 <каталог>"
    exit 1
fi

start_dir="$1"

if [ ! -d "$start_dir" ]; then
    echo "Ошибка: каталог '$start_dir' не существует."
    exit 1
fi

echo "Список файлов в алфавитном порядке (с размерами):"
echo "--------------------------------------------------"

# Временный файл для сохранения результатов
tmpfile=$(mktemp)

# Находим все обычные файлы, выводим "путь<tab>размер", сортируем по пути
find "$start_dir" -type f -printf "%p\t%s\n" | sort > "$tmpfile"

count=$(wc -l < "$tmpfile")

# Вывод с выравниванием через column -t (если доступно)
if command -v column >/dev/null; then
    column -t -s $'\t' "$tmpfile"
else
    cat "$tmpfile"
fi

echo "--------------------------------------------------"
echo "Всего файлов: $count"

rm -f "$tmpfile"
```

```
alexander@alexander-PC:~/Документы/OS/laba3$ ./listfiles.sh /home/alexander/Загрузки/
Список файлов в алфавитном порядке (с размерами):
----------------------------------------------
/home/alexander/Загрузки/1.drawio.svg                                    382200
/home/alexander/Загрузки/1.jpg                                           117145
/home/alexander/Загрузки/answer(1) (2)/styles.css                        288
/home/alexander/Загрузки/answer(1)/styles.css                            288
/home/alexander/Загрузки/answer(1).zip                                   316
/home/alexander/Загрузки/answer(2)/styles.css                            263
/home/alexander/Загрузки/answer(2).zip                                   297
/home/alexander/Загрузки/answer(3)/styles.css                            299
/home/alexander/Загрузки/answer(3).zip                                   335
/home/alexander/Загрузки/answer(4)/styles.css                            525
/home/alexander/Загрузки/answer(4).zip                                   401
/home/alexander/Загрузки/answer(5)/styles.css                            209
/home/alexander/Загрузки/answer(5).zip                                   315
/home/alexander/Загрузки/answer(6) (2)/styles.css                        209
/home/alexander/Загрузки/answer(6)/styles.css                            209
/home/alexander/Загрузки/answer(6).zip                                   315
/home/alexander/Загрузки/answer/styles.css                               787
/home/alexander/Загрузки/answer.zip                                      501
/home/alexander/Загрузки/Diagramma_bez_nazvania_drawio_2(1).svg          420884
/home/alexander/Загрузки/Diagramma_bez_nazvania_drawio_2.svg             420884
/home/alexander/Загрузки/Generated_chart__chart.png.png                  170868
/home/alexander/Загрузки/Group.svg                                       477
/home/alexander/Загрузки/h1-img-1.jpg.jpg                                176217
/home/alexander/Загрузки/h1-rev-icon-1.png.svg                           10263
/home/alexander/Загрузки/header-logo-kropp.png.png                       3771
/home/alexander/Загрузки/Heebo,Yantramanav/Heebo/Heebo-VariableFont_wght.ttf  117508
/home/alexander/Загрузки/Heebo,Yantramanav/Heebo/OFL.txt                 4474
/home/alexander/Загрузки/Heebo,Yantramanav/Heebo/README.txt              2301
/home/alexander/Загрузки/Heebo,Yantramanav/Heebo/static/Heebo-Black.ttf  57160
/home/alexander/Загрузки/Heebo,Yantramanav/Heebo/static/Heebo-Bold.ttf   56812
/home/alexander/Загрузки/Heebo,Yantramanav/Heebo/static/Heebo-ExtraBold.ttf  57032
/home/alexander/Загрузки/Heebo,Yantramanav/Heebo/static/Heebo-ExtraLight.ttf  56356
/home/alexander/Загрузки/Heebo,Yantramanav/Heebo/static/Heebo-Light.ttf  56280
/home/alexander/Загрузки/Heebo,Yantramanav/Heebo/static/Heebo-Medium.ttf 56532
/home/alexander/Загрузки/Heebo,Yantramanav/Heebo/static/Heebo-Regular.ttf  56324
/home/alexander/Загрузки/Heebo,Yantramanav/Heebo/static/Heebo-SemiBold.ttf  56812
/home/alexander/Загрузки/Heebo,Yantramanav/Heebo/static/Heebo-Thin.ttf   55988
/home/alexander/Загрузки/Heebo,Yantramanav/Yantramanav/OFL.txt           4506
/home/alexander/Загрузки/Heebo,Yantramanav/Yantramanav/Yantramanav-Black.ttf  158632
/home/alexander/Загрузки/Heebo,Yantramanav/Yantramanav/Yantramanav-Bold.ttf  157488
/home/alexander/Загрузки/Heebo,Yantramanav/Yantramanav/Yantramanav-Light.ttf  160256
```

```
alexander@alexander-PC:~/Документы/OS/laba3$ ./listfiles.sh /labs
Список файлов в алфавитном порядке (с размерами):
----------------------------------------------
find: '/labs/library': Отказано в доступе
/labs/list                    21958
/labs/tests/test_gornostaev   559
/labs/tests/test_ivan         235
----------------------------------------------
Всего файлов: 3
```

```
alexander@alexander-PC:~/Документы/OS/laba3$ sudo ./listfiles.sh /labs/library
[sudo] пароль для alexander:
Список файлов в алфавитном порядке (с размерами):
--------------------------------------------------
/labs/library/book_gornostaev_1  0
/labs/library/book_gornostaev_2  0
/labs/library/book_gornostaev_3  0
/labs/library/book_gornostaev_4  0
/labs/library/book_gornostaev_5  0
/labs/library/book_ivanov_2      0
--------------------------------------------------
Всего файлов: 6
```

listfiles.sh (вариант 3) – список файлов в поддереве в алфавитном порядке. Принимает имя каталога. Использует find -type f для поиска всех обычных файлов, выводит их пути и размеры (с опцией -printf "%p\t%s\n"). Результат сортируется по пути (sort) и выводится в две колонки (с помощью column -t или просто cat). В конце подсчитывается общее количество файлов.

Часть 2. Компилятор GCC. Оптимизация

Задание. Скрипт для компиляции, замера времени и размера.

```
alexander@alexander-PC:~/Документы/OS/laba3$ nano compile_and_measure.sh
```

```
  GNU nano 7.2                                                          c
#!/bin/bash

if [ $# -lt 1 ]; then
    echo "Использование: $0 <исходный_файл.c> [ключи_оптимизации...]"
    exit 1
fi

source_file="$1"
shift
opt_keys="$@"

if [ ! -f "$source_file" ]; then
    echo "Ошибка: файл '$source_file' не найден."
    exit 1
fi

output_file="./temp_prog_$$"

echo "Компиляция $source_file с ключами: ${opt_keys:-без ключей (по умолчанию -O0)}"
gcc -Wall $opt_keys "$source_file" -o "$output_file" 2> compile_errors.log

if [ $? -ne 0 ]; then
    echo "Ошибка компиляции. Подробности в compile_errors.log"
    exit 1
fi

# Размер исполняемого файла
size=$(stat -c %s "$output_file" 2>/dev/null || wc -c < "$output_file" 2>/dev/null)
echo "Размер исполняемого файла: $size байт"

# Измерение времени выполнения
if command -v /usr/bin/time >/dev/null; then
    real_time=$(/usr/bin/time -f "%e" "$output_file" 2>&1 >/dev/null)
    echo "Время выполнения (real): $real_time сек"
else
    echo "Измерение времени (встроенный time):"
    time "$output_file"
fi

rm -f "$output_file"
```

Для теста создадим файл test.c, в котором будет код на C

```
alexander@alexander-PC:~/Документы/OS/laba3$ nano test.c
```

```
  GNU nano 7.2                                    test.c
#include <stdio.h>

double powern(double d, unsigned n) {
    double x = 1.0;
    for (unsigned j = 1; j <= n; j++)
        x *= d;
    return x;
}

int main(void) {
    double sum = 0.0;
    for (unsigned i = 1; i <= 100000000; i++) {
        sum += powern(i, i % 5);
    }
    printf("sum = %g\n", sum);
    return 0;
}
```

```
alexander@alexander-PC:~/Документы/OS/laba3$ ./compile_and_measure.sh test.c
Компиляция test.c с ключами: без ключей (по умолчанию -O0)
Размер исполняемого файла: 15992 байт
Время выполнения (real): 8.17 сек
Ключи оптимизации: без ключей (эквивалент -O0)
alexander@alexander-PC:~/Документы/OS/laba3$ ./compile_and_measure.sh test.c -O1Компиляция test.c с ключами: -O1
Размер исполняемого файла: 16000 байт
Время выполнения (real): 2.35 сек
Ключи оптимизации: -O1
alexander@alexander-PC:~/Документы/OS/laba3$ ./compile_and_measure.sh test.c -O2Компиляция test.c с ключами: -O2
Размер исполняемого файла: 16000 байт
Время выполнения (real): 1.96 сек
Ключи оптимизации: -O2
alexander@alexander-PC:~/Документы/OS/laba3$ ./compile_and_measure.sh test.c -O3Компиляция test.c с ключами: -O3
Размер исполняемого файла: 16000 байт
Время выполнения (real): 1.79 сек
Ключи оптимизации: -O3
alexander@alexander-PC:~/Документы/OS/laba3$ ./compile_and_measure.sh test.c -OsКомпиляция test.c с ключами: -Os
Размер исполняемого файла: 16000 байт
Время выполнения (real): 10.00 сек
Ключи оптимизации: -Os
```

```
alexander@alexander-PC:~/Документы/OS/laba3$ ./compile_and_measure.sh test.c -funroll-loops
Компиляция test.c с ключами: -funroll-loops
Размер исполняемого файла: 15992 байт
Время выполнения (real): 8.72 сек
Ключи оптимизации: -funroll-loops
```

```
alexander@alexander-PC:~/Документы/OS/laba3$ ./compile_and_measure.sh test.c
Компиляция test.c с ключами: без ключей (по умолчанию -O0)
Размер исполняемого файла: 15992 байт
Время выполнения (real): 8.73 сек
Ключи оптимизации: без ключей (эквивалент -O0)
alexander@alexander-PC:~/Документы/OS/laba3$ ./compile_and_measure.sh test.c
Компиляция test.c с ключами: без ключей (по умолчанию -O0)
Размер исполняемого файла: 15992 байт
Время выполнения (real): 7.39 сек
Ключи оптимизации: без ключей (эквивалент -O0)
```

```
alexander@alexander-PC:~/Документы/OS/laba3$ ./compile_and_measure.sh test.c -O1
Компиляция test.c с ключами: -O1
Размер исполняемого файла: 16000 байт
Время выполнения (real): 2.16 сек
Ключи оптимизации: -O1
alexander@alexander-PC:~/Документы/OS/laba3$ ./compile_and_measure.sh test.c -O1
Компиляция test.c с ключами: -O1
Размер исполняемого файла: 16000 байт
Время выполнения (real): 2.25 сек
Ключи оптимизации: -O1
alexander@alexander-PC:~/Документы/OS/laba3$ ./compile_and_measure.sh test.c -O1
Компиляция test.c с ключами: -O1
Размер исполняемого файла: 16000 байт
Время выполнения (real): 2.14 сек
Ключи оптимизации: -O1
alexander@alexander-PC:~/Документы/OS/laba3$ ./compile_and_measure.sh test.c -O1
Компиляция test.c с ключами: -O1
Размер исполняемого файла: 16000 байт
Время выполнения (real): 2.15 сек
Ключи оптимизации: -O1
alexander@alexander-PC:~/Документы/OS/laba3$ ./compile_and_measure.sh test.c -O2Компиляция test.c с ключами: -O2
Размер исполняемого файла: 16000 байт
Время выполнения (real): 1.95 сек
Ключи оптимизации: -O2
alexander@alexander-PC:~/Документы/OS/laba3$ ./compile_and_measure.sh test.c -O2Компиляция test.c с ключами: -O2
Размер исполняемого файла: 16000 байт
Время выполнения (real): 1.98 сек
Ключи оптимизации: -O2
alexander@alexander-PC:~/Документы/OS/laba3$ ./compile_and_measure.sh test.c -O2Компиляция test.c с ключами: -O2
Размер исполняемого файла: 16000 байт
Время выполнения (real): 2.06 сек
Ключи оптимизации: -O2
alexander@alexander-PC:~/Документы/OS/laba3$ ./compile_and_measure.sh test.c -O2Компиляция test.c с ключами: -O2
Размер исполняемого файла: 16000 байт
Время выполнения (real): 1.96 сек
Ключи оптимизации: -O2
alexander@alexander-PC:~/Документы/OS/laba3$ ./compile_and_measure.sh test.c -O3Компиляция test.c с ключами: -O3
Размер исполняемого файла: 16000 байт
Время выполнения (real): 1.84 сек
Ключи оптимизации: -O3
alexander@alexander-PC:~/Документы/OS/laba3$ ./compile_and_measure.sh test.c -O3Компиляция test.c с ключами: -O3
Размер исполняемого файла: 16000 байт
Время выполнения (real): 1.79 сек
```

```
alexander@alexander-PC:~/Документы/OS/laba3$ ./compile_and_measure.sh test.c -Os
Компиляция test.c с ключами: -Os
Размер исполняемого файла: 16000 байт
Время выполнения (real): 10.10 сек
Ключи оптимизации: -Os
alexander@alexander-PC:~/Документы/OS/laba3$ ./compile_and_measure.sh test.c -Os
Компиляция test.c с ключами: -Os
Размер исполняемого файла: 16000 байт
Время выполнения (real): 6.71 сек
Ключи оптимизации: -Os
alexander@alexander-PC:~/Документы/OS/laba3$ ./compile_and_measure.sh test.c -Os
Компиляция test.c с ключами: -Os
Размер исполняемого файла: 16000 байт
Время выполнения (real): 6.71 сек
Ключи оптимизации: -Os
```

Среднее время выполнения с учётом погрешности:

O0: 8 ± 1.5 сек

O1: 2.35 ± 0.2 сек

O2: 1.96 ± 0.1 сек

O3: 1.79 ± 0.14 сек

Os: 10 ± 4 сек

На основании представленных данных заметим, что на модели процессора AMD Ryzen™ 5 4600H with Radeon™ Graphics × 12 наибольший рост производительности достигается при ключе -O3. Важное замечание: оптимизация не обязательно увеличит производительность программы.

compile_and_measure.sh – компиляция программы на C и замер времени/размера.

Принимает имя исходного файла и опциональные ключи оптимизации. Компилирует программу с помощью gcc (с ключами -Wall и переданными опциями), создаёт временный исполняемый файл. Измеряет его размер командой stat (или wc -c). Замеряет время выполнения через /usr/bin/time -f "%e" (или встроенный time), выводит реальное время. В конце удаляет временный файл и показывает использованные ключи оптимизации.

Ключи оптимизации

- **-O0** (по умолчанию) – отключает все оптимизации. Каждая инструкция исходного кода напрямую соответствует машинным командам. Это упрощает отладку (можно ставить точки останова, смотреть значения переменных), но программа работает медленно и занимает больше места.
- **Базовая оптимизация**

- **-O1** (или просто -O) – включает набор простых оптимизаций, не требующих больших затрат времени компиляции. Они ускоряют программу без существенного увеличения размера кода.
- **Оптимизация для финальной сборки**
- **-O2** – включает большинство распространённых оптимизаций, включая планирование инструкций, оптимизацию циклов и т.д. Размер исполняемого файла почти не растёт, но скорость выполнения заметно повышается. Рекомендуется для готовых программ.
- **Максимальная оптимизация скорости**
- **-O3** – добавляет более агрессивные методы: подстановку функций (инлайнинг), векторизацию и другие. Может значительно ускорить программу, но иногда увеличивает размер кода и даже может снизить производительность из-за промахов кэша.
- **Оптимизация по размеру**
- **-Os** – компилятор старается сделать исполняемый файл как можно меньше (например, для встраиваемых систем). При этом скорость может немного упасть, но программа занимает меньше места на диске и в памяти.
- **Специализированные оптимизации**
- **-funroll-loops** – развёртка циклов: компилятор дублирует тело цикла, чтобы уменьшить количество проверок условия. Это ускоряет выполнение циклов, но сильно увеличивает объём кода.

Вывод

В ходе выполнения лабораторной работы мы изучили программные средства языка Shell (структуру команды, группирование команд, перенаправление ввода-вывода, конвейер команд, Shell-переменные, макроподстановка результатов в Shell-командах, программные конструкции).